

1.4. Effacer le mot rangé sous un nom

Le compilateur connaît la longueur de MOT : on l'efface par son nom.

1. Ce qu'il faut comprendre

`effacer(5, 6, MOT)` efface les sept cases de `MOT`. Si l'on change le mot, l'effacement suit tout seul.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1299 octets** de cartouche.

1. UN TEXTE NOMMÉ — MOT

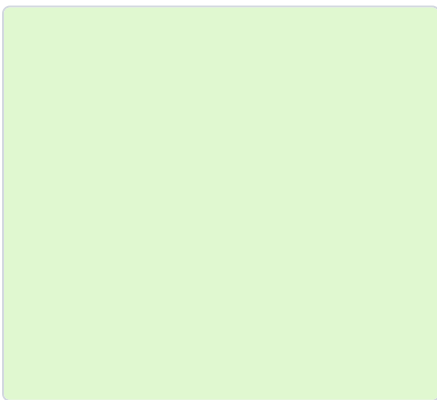
```
const char MOT[] = "BONJOUR";
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(5, 6, MOT);  
    effacer(5, 6, MOT);  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

« BONJOUR » est écrit puis effacé : l'écran reste vide.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les sept cases de BONJOUR sont vides

✓ et l'écran est allumé

Ensuite : **1.5. La place du mot dans deux variables** — niveau 1 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++